

Part 6: Bottom-Up Thinking

Returning information upward instead of recomputing it

The idea in one line

Solve the left subtree, solve the right subtree, combine, and hand the result to your parent.

That is all “bottom-up tree DP” means. It is the postorder position from Part 2 used deliberately. Once you can design **what** to return, you can solve diameter, balanced tree, maximum path sum, largest BST subtree, and most Medium tree problems with the same six lines.

```
solve(node)
    L = solve(node.left)
    R = solve(node.right)
    combine L, R and node
    return the result upward
```

Where the naive approach goes wrong

Here is a concrete, honest example. Suppose we want, for every node, the height of the subtree rooted at that node. The direct approach:

```
int height(TreeNode* root) {
    if (root == NULL) return 0;
    return 1 + max(height(root->left), height(root->right));
}

void printAllHeights(TreeNode* root) {
    if (root == NULL) return;
    cout << root->val << " has height " << height(root) << "\n";    // full walk
    printAllHeights(root->left);
    printAllHeights(root->right);
}
```

It works. But look at the cost. `printAllHeights` visits N nodes, and at each node it calls `height`, which itself walks the whole subtree below that node.

On a skewed tree of N nodes:

```
node 1: height() walks N nodes
node 2: height() walks N-1 nodes
```

node 3: height() walks N-2 nodes

...

total = N + (N-1) + ... + 1 = N(N+1)/2 = $O(N^2)$

On a balanced tree the same reasoning gives $O(N \log N)$. Either way, work is being repeated. When height(1) ran, it already computed the height of node 2 internally, then threw it away, and then we computed it again from scratch.

That throwing away is the bug. The information was already in our hands during the recursion, on the way back up.

The fix: return it instead of recomputing it

```
int height(TreeNode* root) {
    if (root == NULL) return 0;
    int L = height(root->left);    // the left subtree just told me its height
    int R = height(root->right);   // so did the right one
    cout << root->val << " has height " << 1 + max(L, R) << "\n";
    return 1 + max(L, R);
}
```

One traversal. $O(N)$. Each node computes its own answer in $O(1)$ using the two numbers its children returned.

The general principle:

If your outer function calls an inner recursive function at every node, you are almost certainly recomputing something that the recursion could have returned. Merge them into one pass.

This exact rewrite converts the naive diameter solution (Part 7) and the naive balanced check (Part 8) from $O(N^2)$ to $O(N)$. Those two problems exist on LeetCode largely to teach this rewrite.

Designing the return value

The whole skill is answering this question:

What does my parent need to know about me?

Not “what is the final answer”, but “what must travel up one edge”. Some examples:

Problem	What each node returns upward
Height	height of my subtree
Count nodes	number of nodes in my subtree
Balanced check	my height, or a flag saying I am already broken
Diameter	my height (the diameter is tracked separately)
Maximum path sum	best downward path sum starting at me
Largest BST subtree	(is my subtree a BST, min, max, size)
Sum of subtree values	sum of my subtree

Notice a recurring split in the last few rows. There are two different quantities:

- **the answer through me**, which may involve both of my children, and
- **the value I can give my parent**, which can only continue through **one** child, because a path cannot go down the left, come back, and go down the right.

Diameter and maximum path sum are the classic examples, and confusing these two is the number one error in this whole topic. We deal with it properly in Part 7.

Two ways to carry extra information

Way 1: return a small struct

Clean, explicit, and easy to explain in an interview.

```
struct Info {
    int height;
    int count;
};

Info solve(TreeNode* root) {
    if (root == NULL) return {0, 0};

    Info L = solve(root->left);
    Info R = solve(root->right);

    Info me;
    me.height = 1 + max(L.height, R.height);
    me.count = 1 + L.count + R.count;
    return me;
}
```

Use this when two or more values genuinely need to travel upward together.

Way 2: return one value, keep the answer in a reference or member

Shorter, and it is what most accepted LeetCode solutions look like.

```
class Solution {
    int best = 0; // the global answer

    int height(TreeNode* root) { // returns only the height
        if (root == NULL) return 0;
        int L = height(root->left);
        int R = height(root->right);
        best = max(best, L + R); // update the answer at this node
        return 1 + max(L, R); // hand only the height upward
    }
public:
    int diameterOfBinaryTree(TreeNode* root) {
        best = 0;
        height(root);
    }
}
```

```

    return best;
}
};

```

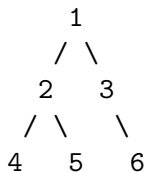
Read that carefully, because it contains the entire lesson of this part:

- $L + R$ is **the answer through this node**. It uses both children. It is recorded, not returned.
- $1 + \max(L, R)$ is **what the parent can use**. It continues through one child only.

If you ever find yourself wanting to return $L + R$ upward, stop. That would be claiming the parent can extend a path that already turned a corner, which is geometrically impossible.

Always reset the member variable at the start of the public function, since LeetCode reuses the object across test cases.

A worked example: the whole pattern on our tree



Compute, in one postorder pass, the height and node count of every subtree.

```

solve(4): L={0,0} R={0,0}  -> {height 1, count 1}
solve(5): L={0,0} R={0,0}  -> {height 1, count 1}
solve(2): L={1,1} R={1,1}  -> {height 2, count 3}
solve(6): L={0,0} R={0,0}  -> {height 1, count 1}
solve(3): L={0,0} R={1,1}  -> {height 2, count 2}
solve(1): L={2,3} R={2,2}  -> {height 3, count 6}

```

Six lines, six nodes, each computed exactly once. Nothing was recomputed. That is what $O(N)$ looks like.

Also notice the order the lines completed in: 4, 5, 2, 6, 3, 1, the postorder. Bottom-up and postorder are the same thing described from two directions.

When bottom-up is not the answer

Not everything flows upward. Some problems need information handed **downward** from the parent:

- “Print all root-to-leaf paths”: each child needs the path so far.
- “Path sum equals target”: each child needs the remaining target.
- “Is this a valid BST”: each child needs the allowed (min, max) range.
- “Depth of each node”: each child needs the parent’s depth.

That is a preorder shape, and it is normally written with an extra parameter:

```
void dfs(TreeNode* root, int depthSoFar) {  
    if (root == NULL) return;  
    dfs(root->left, depthSoFar + 1);  
    dfs(root->right, depthSoFar + 1);  
}
```

So the first question on any tree problem is:

Does information flow down, up, or both?

Down means preorder and extra parameters. Up means postorder and a return value. Both means you do both in the same function, which is completely normal: descend carrying context, return summaries as you unwind.

Complexity, and why it comes out to $O(N)$

For a single postorder pass:

- Each node is entered exactly once.
- At each node the combine step is $O(1)$, assuming you are combining a constant number of integers.
- Total: **$O(N)$ time.**
- Recursion depth equals the height: **$O(H)$ space**, which is $O(\log N)$ balanced and $O(N)$ skewed.

If your combine step is not $O(1)$, the total changes. For example, if at each node you merge two maps of size up to N , the cost is not $O(N)$ any more. Keep the combine cheap.

Common mistakes

- **Recomputing height inside another recursion.** The $O(N^2)$ trap.
- **Returning the “through me” value upward.** Diameter and maximum path sum both break this way, and they break silently: the answer is only wrong on some trees, so a couple of examples may still pass.
- **Forgetting to reset a member variable** between runs.
- **Choosing a return type before understanding the problem.** Decide what the parent needs first, then pick the type.
- **Using a struct when one int would do**, or a single int when the problem really needs two values. Both directions cost you time in an interview.
- **Doing the combine before the recursive calls.** At that moment you know nothing about the subtrees.

Check yourself

1. Why is calling `height()` at every node $O(N^2)$ in the worst case? Which tree shape causes it?
2. What is the difference between “the answer through this node” and “the value returned to the parent”?
3. In the diameter code above, why is `L + R` recorded but `1 + max(L, R)` returned?

4. Give a problem where information must flow downward, and say how you would carry it.
5. You want the size of the largest subtree that contains only even values. What should each node return to its parent?
6. Why is the time complexity of a single postorder pass $O(N)$ and not $O(N \log N)$?